AGENTTRACE: CAUSAL GRAPH TRACING FOR ROOT CAUSE ANALYSIS IN DEPLOYED MULTI-AGENT SYSTEMS

Zhaohui Geoffrey Wang
USC Viterbi School of Engineering
zwang000@usc.edu
ORCID: 0009-0006-1187-1903

arXiv:2603.14688v2 [cs.LG] 27 Mar 2026

ABSTRACT

As multi-agent AI systems are increasingly deployed in real-world settings—from automated customer support to DevOps remediation—failures become harder to diagnose due to cascading effects, hidden dependencies, and long execution traces. We present AGENTTRACE, a lightweight causal tracing framework for post-hoc failure diagnosis in deployed multi-agent workflows. AGENTTRACE reconstructs causal graphs from execution logs, traces backward from error manifestations, and ranks candidate root causes using interpretable structural and positional signals—without requiring LLM inference at debugging time. Across a diverse benchmark of multi-agent failure scenarios designed to reflect common deployment patterns, AGENTTRACE localizes root causes with high accuracy and sub-second latency, significantly outperforming both heuristic and LLM-based baselines. Our results suggest that causal tracing provides a practical foundation for improving the reliability and trustworthiness of agentic systems in the wild.

1 INTRODUCTION

Multi-agent systems powered by large language models (LLMs) have emerged as a powerful paradigm for solving complex tasks through agent collaboration (Wu et al., 2023; Hong et al., 2024). In these systems, specialized agents—such as planners, coders, reviewers, and executors—coordinate through message passing to accomplish goals that would be difficult for a single agent.

In deployed agent systems—such as automated customer support, DevOps remediation, or research assistants—failures often surface far downstream from their root causes. By the time an error is observed, multiple agents may have already acted on corrupted assumptions, making manual debugging slow and unreliable. The distributed and emergent nature of these workflows means that traditional debugging approaches, which examine individual components in isolation, fail to capture the cross-agent causal dependencies that lead to system-level failures.

We propose AGENTTRACE, a lightweight framework that addresses this challenge through three key contributions:

1. **Causal Graph Construction**: We model multi-agent execution as a directed graph where nodes represent agent actions and edges capture information flow and causal dependencies.
2. **Backward Tracing Algorithm**: Starting from the point of error manifestation, we trace backward through the causal graph to identify all potentially relevant upstream decisions.
3. **Empirical Study of Failure Localization**: We demonstrate that lightweight causal tracing with interpretable positional and structural features can substantially improve debugging accuracy and latency in settings that resemble real-world agent deployments—without requiring expensive LLM inference at debugging time.

While our benchmark scenarios are synthetically constructed, they are designed to capture common failure patterns observed in deployed multi-agent systems, such as early planning errors cascading

AGENTTRACE：已部署多智能体系统中的因果图追踪用于根本原因分析

Zhaohui Geoffrey Wang 南加州大学维
特比工程学院 zwang000@usc.edu
ORCID: 0009-0006-1187-1903

arXiv:2603.14688v2 [cs.LG] 27 Mar 2026

摘要

随着多智能体人工智能系统在现实场景中日益普及——从自动化客户支持到DevOps修复——由于级联效应、隐藏依赖以及冗长的执行轨迹，故障诊断变得更加困难。我们提出了AGENTTRACE，这是一种用于已部署多智能体工作流中事后故障诊断的轻量级因果追踪框架。AGENTTRACE从执行日志中重建因果图，从错误表现处进行反向追踪，并利用可解释的结构和位置信号对候选根本原因进行排序——无需在调试时进行大语言模型推理。在一个旨在反映常见部署模式的多智能体故障场景基准测试中，AGENTTRACE以高准确率和亚秒级延迟定位根本原因，显著优于启发式和基于大语言模型的基线。我们的结果表明，因果追踪为提高野外智能体系统的可靠性和可信度提供了实用基础。

1 引言

由大语言模型（LLMs）驱动的多智能体系统已成为一种通过智能体协作解决复杂任务的强大范式（Wu等人，2023; Hong等人，2024）。在这些系统中，诸如规划者、编码者、审查者和执行者等专业化智能体通过消息传递进行协调，以完成单个智能体难以实现的目标。

在已部署的智能体系统（例如自动化客户支持、DevOps修复或研究助理）中，故障往往在其根本原因的下游远端显现。当观察到错误时，多个智能体可能已经基于损坏的假设采取了行动，这使得手动调试变得缓慢且不可靠。这些工作流的分布式和涌现特性意味着，传统的调试方法——即孤立地检查单个组件——无法捕捉导致系统级故障的跨智能体因果依赖。

我们提出了AGENTTRACE，一个轻量级框架，通过以下三项关键贡献来解决这一挑战：

1. 因果图构建：我们将多智能体执行建模为一个有向图，其中节点代表智能体动作，边捕捉信息流和因果依赖。2. 反向追踪算法：从错误表现点开始，我们通过因果图向后追踪，以识别所有可能相关的上游决策。3. 故障定位的实证研究：我们证明了，在与真实世界智能体部署相似的环境中，采用可解释的位置和结构特征的轻量级因果追踪，能够显著提升调试的准确率和时延——而无需在调试时进行昂贵的大语言模型推理。

尽管我们的基准测试场景是合成构建的，但它们旨在捕捉在已部署的多智能体系统中观察到的常见故障模式，例如早期规划错误引发的级联故障

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

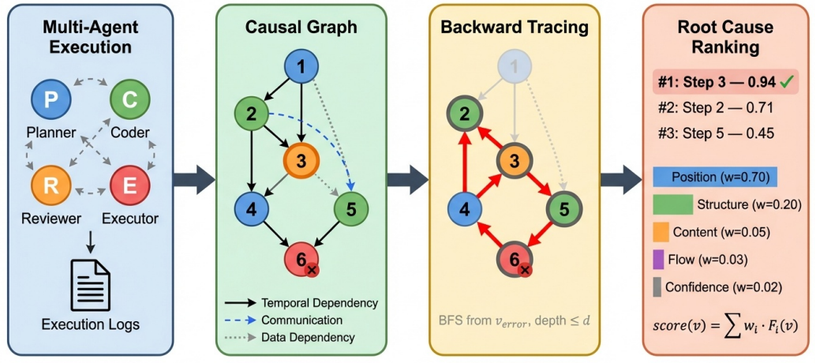


Figure 1: Overview of the AGENTTRACE framework. (a) Execution logs from a multi-agent workflow are collected. (b) A causal graph is constructed with sequential, communication, and data dependency edges. (c) Backward BFS from the error node identifies candidate root causes. (d) Candidates are ranked using a weighted combination of five interpretable feature groups, with position features dominating (w_p=0.70).

through execution. Our evaluation across 550 failure scenarios in 10 domains shows that AGENTTRACE achieves high localization accuracy with sub-second latency, significantly outperforming both heuristic and LLM-based baselines. Figure 1 provides an overview of the framework.

2 RELATED WORK

Multi-Agent Systems. Recent frameworks like AutoGen (Wu et al., 2023) and MetaGPT (Hong et al., 2024) enable sophisticated multi-agent collaboration, often building on reasoning-and-acting paradigms (Yao et al., 2023). However, debugging support in these systems remains limited, typically relying on manual log inspection.

AI System Debugging. Prior work on AI interpretability has focused on neural network internals (Simonyan et al., 2014; Kim et al., 2018) or reasoning chains (Wei et al., 2022). Self-debugging approaches (Chen et al., 2024) use LLMs to identify errors but require expensive inference calls and struggle with cross-agent issues.

Distributed Tracing. Systems like Jaeger (Technologies, 2017) and Zipkin (Twitter, 2012) provide distributed tracing for microservices. We adapt these concepts for multi-agent LLM systems, where "messages" between agents carry semantic content rather than just request metadata.

Root Cause Analysis. Traditional RCA approaches use statistical methods (Soldani & Brogi, 2022) or graph algorithms (Page et al., 1999). Recent work explores LLM-based agents for root cause analysis in cloud systems (Roy et al., 2024), but these methods are not designed for the specific structure of multi-agent workflows.

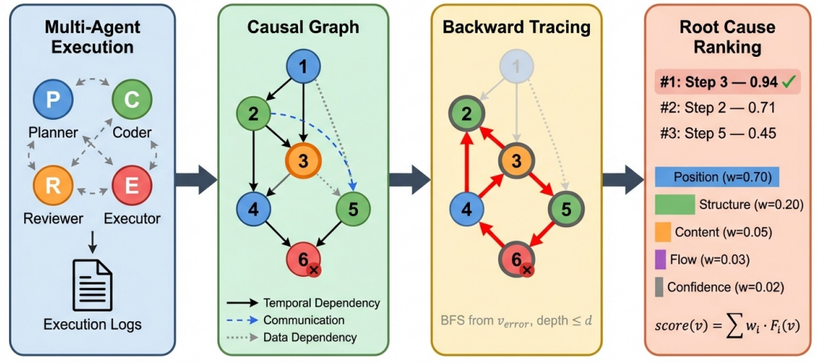


图1：AgentTrace 框架概览。(a) 收集来自多智能体工作流的执行日志。(b) 构建包含顺序型、通信型和数据依赖型边的因果图。(c) 从错误节点进行反向广度优先搜索以识别候选根本原因。(d) 使用五个可解释特征组的加权组合对候选项进行排序，其中位置特征占主导地位（w_p=0.70）。

通过执行实现。我们在10个领域的550个故障场景中进行的评估表明，AGENTTRACE能够以亚秒级延迟实现高定位准确率，显著优于启发式和基于大语言模型的基线。图1提供了该框架的概述。

2 相关工作

多智能体系统。近期框架如 AutoGen（Wu等人，2023）和 MetaGPT（Hong等人，2024）实现了复杂的多智能体协作，通常基于推理-行动范式（Yao et al., 2023）。然而，这些系统中的调试支持仍然有限，通常依赖于手动日志检查。

AI 系统调试。先前关于 AI 可解释性的工作主要关注神经网络内部结构（Simonyan 等，2014；Kim 等，2018）或推理链（Wei 等，2022）。自调试方法（Chen 等，2024）利用大语言模型识别缺陷，但需要昂贵的推理调用，且在处理跨智能体问题时表现不佳。

分布式追踪。Jaeger (Technologies, 2017) 和 Zipkin (推特, 2012) 等系统为微服务提供分布式追踪。我们将这些概念适配到多智能体大语言模型系统，其中智能体之间的"消息"承载的是语义内容，而不仅仅是请求元数据。

根本原因分析。传统的根本原因分析方法采用统计方法（Soldani & Brogi, 2022）或图算法（Page et al., 1999）。近期工作探索了用于云系统根本原因分析的基于大语言模型的智能体（Roy et al., 2024），但这些方法并非针对多智能体工作流的具体结构设计。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

3 AGENTTRACE FRAMEWORK

3.1 PROBLEM DEFINITION

Given a multi-agent execution trace T that terminates in an error state, our goal is to identify the *root cause node*—the earliest decision point whose correction would prevent the error. Formally, let $G = (V, E)$ be a directed acyclic graph where:

- $V = \{v_1, v_2, \ldots, v_n\}$ represents agent actions (tool calls, messages, decisions)
- $E \subseteq V \times V$ represents causal dependencies between actions
- $v_{\text{error}} \in V$ is the node where the error manifests
- $v_{\text{root}} \in V$ is the ground-truth root cause

3.2 CAUSAL GRAPH CONSTRUCTION

We construct the causal graph from execution logs by identifying three types of edges:

Sequential edges connect consecutive actions by the same agent, capturing the agent's reasoning flow.

Communication edges connect message-send events to message-receive events between different agents.

Data dependency edges connect actions that produce data to actions that consume that data, identified through variable reference tracking.

3.3 BACKWARD TRACING

Given an error node v_{error}, we perform breadth-first backward traversal to collect all ancestor nodes within a specified depth limit:

Algorithm 1 Backward Tracing

Require: Error node v_{error}, graph G, max depth d
Ensure: Candidate set C

$C \leftarrow \{v_{\text{error}}\}$
frontier $\leftarrow \{v_{\text{error}}\}$
for $i = 1$ to d **do**
  new_frontier $\leftarrow \emptyset$
  for $v \in$ frontier **do**
    for $u \in$ parents(v) **do**
      if $u \notin C$ **then**
        $C \leftarrow C \cup \{u\}$
        new_frontier $\leftarrow$ new_frontier $\cup \{u\}$
      end if
    end for
  end for
  frontier $\leftarrow$ new_frontier
end for
return C

3.4 NODE RANKING ALGORITHM

We rank candidate nodes using a weighted linear combination of five feature groups:

$$\text{score}(v) = \sum_{i \in \{p,s,c,f,e\}} w_i \cdot F_i(v) \tag{1}$$

3 AGENTTRACE 框架

3.1 问题定义

给定一个以错误状态终止的多智能体执行轨迹 T，我们的目标是识别根因节点——即最早的那个决策点，纠正它可以防止错误的发生。形式上，设$G = (V, E)$为一个有向无环图，其中：

- $V = \{v_1, v_2, \ldots, v_n\}$ 表示智能体动作（工具调用、消息、决策）
- $E \subseteq V \times V$表示动作之间的因果依赖
- v 错误 $\in V$ 是错误表现所在的节点
- v根节点 $\in V$是真实根因

3.2 因果图构建

我们通过从执行日志中识别三种类型的边来构建因果图：

Sequential 边连接同一智能体的连续动作，捕捉该智能体的推理 g
flow.
通信边将不同智能体之间的消息发送事件连接到消息接收事件。

数据依赖边连接产生数据的操作与消费该数据的操作，通过变量引用跟踪进行识别。

3.3 向后追踪

给定一个错误节点 v_{error}，我们执行广度优先向后遍历以收集指定深度限制内的所有祖先节点：

算法 1 向后追踪

要求：错误节点 v_{error}，图 G，最大深度 d
保证：候选集 C

$C \leftarrow \{v_{\text{error}}\}$
前沿 $\leftarrow \{v_{\text{error}}\}$
从 $i = 1$到 d 执行
  新前沿 $\leftarrow \emptyset$
  遍历 $v \in$ 前沿 执行
    遍历 $u \in$ 父节点(v) 执行
      如果 $u \notin C$ **则**
        $C \leftarrow C \cup \{u\}$
        新前沿 $\leftarrow$ 新前沿 $\cup \{u\}$
      结束if
    结束for
  结束for
  前沿 ←新前沿_
结束for
return C

3.4 节点排序算法

我们使用五个特征组的加权线性组合对候选节点进行排序：

$$\text{score}(v) = \sum_{i \in \{p,s,c,f,e\}} w_i \cdot F_i(v) \tag{1}$$

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 1: Complete feature list with computation methods

Group (w_i)	**Feature**	**Formula**	**Range**
Position (0.70)	Normalized Position	$\text{pos}(v)/\lvert V\rvert$	$[0,1]$
	Distance to Error	$d(v, v_{\text{error}})/\max_u d(u, v_{\text{error}})$	$[0,1]$
	Depth Ratio	$\text{depth}(v)/\max_u \text{depth}(u)$	$[0,1]$
	Reverse Position	$1-\text{pos}(v)/\lvert V\rvert$	$[0,1]$
Structure (0.20)	Out-degree	$\lvert\{u:(v,u)\in E\}\rvert/\max_w \lvert\{u:(w,u)\in E\}\rvert$	$[0,1]$
	In-degree	$\lvert\{u:(u,v)\in E\}\rvert/\max_w \lvert\{u:(u,w)\in E\}\rvert$	$[0,1]$
	Betweenness	$\sum_{s\neq v\neq t}\frac{\sigma_{st}(v)}{\sigma_{st}}$ (normalized)	$[0,1]$
	Reachability	$\lvert\text{descendants}(v)\rvert/\lvert V\rvert$	$[0,1]$
Content (0.05)	Error Keywords	$\mathbb{1}[\text{“error”}\in\text{content}(v)]$	$\{0,1\}$
	Uncertainty	$\mathbb{1}[\text{“maybe”}\in\text{content}(v)]$	$\{0,1\}$
	Length Anomaly	$\lvert\text{len}-\mu\rvert/(3\sigma)$ (clipped)	$[0,1]$
	Keyword Density	count(keywords)/len	$[0,1]$
Flow (0.03)	Agent Switch	$\mathbb{1}[\text{agent}(v)\neq\text{agent}(\text{prev}(v))]$	$\{0,1\}$
	Role Criticality	$\text{role_weight}(\text{agent}(v))$	$[0,1]$
	Communication	$\mathbb{1}[\text{type}(v)=\text{“message”}]$	$\{0,1\}$
Confidence (0.02)	Stated Confidence	$\text{conf}(v)$ if available, else 0.5	$[0,1]$
	Hedging Score	$\text{count}(\text{hedge_words})/10$ (clipped)	$[0,1]$

Each group score $F_i(v)$ is the mean of its constituent normalized features:

$$F_i(v)=\frac{1}{|G_i|}\sum_{j\in G_i}\hat{f}_j(v),\quad \hat{f}_j(v)=\frac{f_j(v)-\min_u f_j(u)}{\max_u f_j(u)-\min_u f_j(u)+\epsilon} \tag{2}$$

where $\epsilon=10^{-8}$ prevents division by zero. The weights w_i are determined via grid search on a held-out validation set of 50 scenarios (see Appendix B). Table 1 lists all 17 features across the five groups.

4 Benchmark

4.1 Scenario Generation

We created a benchmark of 550 synthetic multi-agent failure scenarios across 10 domains (Table 2). Each scenario includes a complete execution trace with 8–15 agent actions, a systematically injected bug at a specific node, and ground-truth annotation of the root cause node.

Table 2: Benchmark domains and scenario counts (550 total)

Domain	**N**	**Domain**	**N**
Software Development	52	Healthcare Coordination	60
Customer Service	51	Legal Document Analysis	60
Research Analysis	51	Educational Tutoring	60
Planning & Scheduling	46	Financial Advisory	60
Financial Trading	50	DevOps Automation	60

4.2 Bug Injection and Ground Truth

We inject five categories of bugs: **logic errors** (incorrect conditionals, 30%), **communication failures** (message misinterpretation, 20%), **data corruption** (incorrect transformation, 20%), **missing validation** (skipped verification, 16%), and **role confusion** (acting outside expertise, 14%).

Bug injection follows a five-step process: (1) generate a correct execution trace from domain templates; (2) select a bug location (early steps 2–3: 60%, middle steps 4–6: 30%, late steps 7+: 10%);

表1：包含计算方法的完整特征列表

组（w_i）	**特征**	**公式**	**范围**
位置 (0.70)	归一化位置	$\text{pos}(v)/\lvert V\rvert$	$[0,1]$
	距ERROR的距离	$d(v, v_{\text{error}})/\max_u d(u, v_{\text{error}})$	$[0,1]$
	深度比	深度(v)/最大值$_u$ 深度(u)	$[0,1]$
	反向位置	$1-\text{pos}(v)/\lvert V\rvert$	$[0,1]$
结构 (0.20)	出度	$\lvert\{u:(v,u)\in E\}\rvert/\max_w \lvert\{u:(w,u)\in E\}\rvert$	$[0,1]$
	入度	$\lvert\{u:(u,v)\in E\}\rvert/\max_w \lvert\{u:(u,w)\in E\}\rvert$	$[0,1]$
	介数中心性	$\sum_{s\neq v\neq t}\frac{\sigma_{st}(v)}{\sigma_{st}}$（归一化）	$[0,1]$
	可达性	$\lvert$后代$(v)\rvert/\lvert V\rvert$	$[0,1]$
内容（0.05）	错误关键词	$\mathbb{1}[\text{“error”}\in$内容$(v)]$	$\{0,1\}$
	不确定性	$\mathbb{1}[\text{“maybe”}\in$内容$(v)]$	$\{0,1\}$
	长度异常	$\lvert\text{len}-\mu\rvert/(3\sigma)$（截断）	$[0,1]$
	关键词密度	count(keywords)/len	$[0,1]$
流程 (0.03)	智能体切换	$\mathbb{1}[$智能体$(v)\neq$智能体（prev(v)）]	$\{0,1\}$
	角色关键性	角色权重(agent(v))	$[0,1]$
	通信型	$\mathbb{1}[$类型$(v)=$“消息”]	$\{0,1\}$
置信度 (0.02)	陈述置信度	conf(v)，如可用，否则为0.5	$[0,1]$
	模糊化分数	count(模糊词)/10(截断)	$[0,1]$

每个组的得分 $F_i(v)$ 是其归一化特征的均值：

$$F_i(v)=\frac{1}{|G_i|}\sum_{j\in G_i}\hat{f}_j(v),\quad \hat{f}_j(v)=\frac{f_j(v)-\min_u f_j(u)}{\max_u f_j(u)-\min_u f_j(u)+\epsilon} \tag{2}$$

其中 $\epsilon=10^{-8}$ 防止除以零。权重 w_i 通过在包含50个场景的留出验证集上进行网格搜索确定（见附录B）。表1列出了五个组中的全部17个特征。

4 基准测试

4.1 场景生成

我们创建了一个包含550个合成的多智能体故障场景的基准测试，覆盖10个领域（表2）。每个场景包含一条完整的执行轨迹，具有8–15个智能体动作、在特定节点系统性注入的缺陷，以及对根因节点的真实标注。

表2: 基准测试的领域和场景数量（总计550个）

领域	**N**	**领域**	**N**
软件开发	52	医疗协调	60
客户服务	51	法律文件分析	60
研究分析	51	教育辅导	60
计划与调度	46	财务咨询	60
金融交易	50	DevOps自动化	60

4.2 缺陷注入与基准数据

我们注入五类缺陷：逻辑错误（条件错误，30%）、通信故障（消息误解，20%）、数据损坏（转换错误，20%）、缺失验证（跳过验证，16%）以及角色混淆（超出专业范围的行为，14%）。

缺陷注入遵循一个五步骤流程：（1）从领域模板生成正确的执行轨迹；（2）选择缺陷位置（早期步骤 2–3：60%，中间步骤 4–6：30%，后期步骤 7+：10%）；

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

(3) assign a bug type; (4) inject erroneous content (e.g., flipped comparison operators, message truncation, swapped variable names); (5) propagate cascading effects to downstream steps. To prevent methods from exploiting explicit bug markers, we also produce a blind version with anonymized IDs and separated ground truth.

Each scenario's ground truth was established through: (1) controlled bug injection at a known step, (2) verification that removing the bug prevents the error, and (3) confirmation that the bug's effects propagate to the error node. The constructed causal graphs have on average 10.8 nodes (σ=2.1) and 14.2 edges (σ=3.5), with edges predominantly sequential (61%), followed by communication (27%) and data dependency (12%).

5 EXPERIMENTS

5.1 EVALUATION METRICS

- **Hit@k**: Proportion of scenarios where ground truth appears in top-k predictions
- **Mean Reciprocal Rank (MRR)**: Average of $1/\text{rank}$

5.2 BASELINES

- **Random**: Uniformly random node selection
- **First Node**: Always select the first node
- **Last Node**: Select the node immediately before error
- **LLM Analysis**: GPT-4 with full trace and prompt engineering (details in Appendix E)

5.3 MAIN RESULTS

Table 3 shows that AGENTTRACE significantly outperforms all baselines. McNemar's test confirms significance ($p < 10^{-10}$) for all comparisons, with Cohen's h = 0.77 vs. LLM Analysis indicating a large practical difference.

Table 3: Root cause localization accuracy and statistical significance on the 550-scenario benchmark. 95% CIs from bootstrap resampling (B=10,000). All p-values $< 10^{-10}$ (McNemar's test).

Method	Hit@1	Hit@3	MRR	χ^2	Cohen's h
Random	9.1%	27.3%	0.18	459.2	2.41
First Node	3.6%	10.9%	0.07	497.0	2.72
Last Node	12.7%	38.2%	0.25	436.5	2.29
LLM Analysis	68.5%	81.4%	0.74	102.5	0.77
AGENTTRACE	**94.9%** [92.9, 96.7]	**98.4%** [96.9, 99.4]	**0.97**	–	–

5.4 ABLATION STUDY

Table 4 shows Hit@1 for all feature group combinations and the sensitivity to the position weight w_p. Position features alone reach 87.3%, while each additional group provides incremental gains. The optimal $w_p = 0.70$ balances predictive power with discriminative information from other groups.

5.5 ROBUSTNESS ANALYSIS

Table 5 stratifies results across three dimensions, demonstrating that AGENTTRACE is robust to bug type (93.6–95.8% Hit@1), trace length (93.9–95.5%), and bug position (92.7–96.1%).

5.6 ILLUSTRATIVE EXAMPLES

We present two representative scenarios to illustrate how AGENTTRACE operates.

（3）分配缺陷类型；（4）注入错误内容（例如，反转的比较运算符、消息截断、交换的变量名）；（5）将级联效应传播到下游步骤。为防止方法利用显式缺陷标记，我们还会生成一个盲测版本，其中包含匿名化标识符和分离的真实标签。

每个场景的真实标签通过以下方式确定：(1) 在已知步骤进行受控的缺陷注入，(2) 验证移除该缺陷可防止错误发生，以及 (3) 确认该缺陷的影响传播至错误节点。所构建的因果图平均包含 10.8 个节点（σ=2.1）和 14.2 条边（σ=3.5），其中边以顺序型（61%）为主，其次是通信型（27%）和数据依赖型（12%）。

5 实验

5.1 评估指标

- Hit@k：真实标签出现在前 k 个预测中的场景比例
- **平均倒数排名 (MRR)：** $1/$**rank 的平均值**

5.2 基线

- 随机：均匀随机节点选择
- 首个节点：始终选择首个节点
- 末个节点：选择紧邻ERROR之前的节点
- 大语言模型分析：采用GPT-4及完整轨迹与提示工程（细节见附录E）

5.3 主要结果

表3表明，AGENTTRACE显著优于所有基线。麦纳玛检验证实了所有比较的显著性（$p < 10^{-10}$），其中与LLM分析相比，科恩h为 = 0.77，表明存在较大的实际差异。

表 3：550场景基准测试中的根本原因定位准确率及统计显著性。通过自举重采样（B=10,000）得到的95%置信区间。所有 p值 $< 10^{-10}$（麦纳玛检验）。

方法	Hit@1	Hit@3	MRR	χ^2	科恩h
随机	9.1%	27.3%	0.18	459.2	2.41
首个节点	3.6%	10.9%	0.07	497.0	2.72
末个节点	12.7%	38.2%	0.25	436.5	2.29
LLM分析	68.5%	81.4%	0.74	102.5	0.77
AGENTTRACE	94.9% [92, 96.7]	98.4% [96.9, 99.4]	**0.97**	–	–

5.4 消融研究

表4展示了所有特征组合的Hit@1以及对位置权重 w_p的敏感性。仅使用位置特征时，Hit@1达到87.3%，而每增加一个特征组都会带来额外的提升。最优的 $w_p = 0.70$ 在预测能力与其他特征组的判别信息之间取得平衡。

5.5 鲁棒性分析

表5从三个维度对结果进行了分层，表明AGENTTRACE对缺陷类型（93.6–95.8% Hit@1）、轨迹长度（93.9–95.5%）以及错误位置（92.7–96.1%）均具有鲁棒性。

5.6 示例

我们提出两个代表性场景来说明 AGENTTRACE 是如何运行的。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 4: Ablation study. *Left*: Hit@1 with different feature group combinations (P=Position, S=Structure, C=Content, F=Flow, E=Confidence). *Right*: Sensitivity of Hit@1 to position weight w_p.

Features	**Hit@1**	Δ	w_p	**Hit@1**
All (P+S+C+F+E)	94.9%	–	0.50	91.3%
P only	87.3%	-7.6	0.60	93.5%
S only	34.5%	-60.4	**0.70**	**94.9%**
C only	28.7%	-66.2	0.80	93.8%
F only	15.2%	-79.7	0.90	91.1%
E only	12.1%	-82.8		
P+S	92.4%	-2.5		
P+C	90.9%	-4.0		
P+F / P+E	89.5 / 88.7%	-5.4 / -6.2		
S+C / S+F	45.6 / 38.2%	-49.3 / -56.7		
P+S+C	93.8%	-1.1		
P+S+C+F	94.5%	-0.4		

Table 5: Robustness analysis: Hit@1 and MRR stratified by bug type, trace length, and bug position.

Stratification	**Category**	**Hit@1**	**MRR**
Bug Type	Logic Error	95.8%	0.98
	Communication Failure	93.6%	0.96
	Data Corruption	94.5%	0.97
	Missing Validation	94.3%	0.96
	Role Confusion	94.8%	0.97
Trace Length	8–9 steps	95.5%	0.97
	10–11 steps	95.0%	0.97
	12–13 steps	94.2%	0.96
	14–15 steps	93.9%	0.96
Bug Position	Early (steps 2–3)	96.1%	0.98
	Middle (steps 4–6)	93.3%	0.96
	Late (steps 7+)	92.7%	0.95

Example 1: Software Development. *Task:* Implement average calculation.

```
Step 1 [Planner]: Analyze requirements
  -> "Sum all numbers, count elements, divide"
Step 2 [Coder]: Write implementation
  -> "def average(nums): ..."
Step 3 [Coder]: Add edge case handling  [BUG]
  -> "if nums = []:  # BUG: = instead of =="
Step 4 [Reviewer]: Review code -> "Code looks correct."
Step 5 [Executor]: Run tests  [ERROR]
  -> "SyntaxError: invalid syntax at line 2"
```

Root cause: Step 3 (logic error). AGENTTRACE correctly identifies Step 3; the LLM baseline selects Step 5 (error node), illustrating its tendency to select the error manifestation rather than tracing back.

Example 2: Research Analysis. *Task:* Analyze transformer architecture publications.

```
Step 1 [Searcher]: Query databases -> "150 papers"
Step 2 [Searcher]: Filter -> "25 relevant papers"
Step 3 [Analyzer]: Extract findings  [BUG]
  -> "Attention being replaced by MLPs"
     (actually: attention being enhanced)
Step 4 [Synthesizer]: "Field moving away from
```


表4：消融研究。左：使用不同特征组合（P=位置、S=结构、C=内容、F=流程、E=置信度）时的Hit@1。右：Hit@1对位置权重w_p的敏感性。

特征	**Hit@1**	Δ	w_p	**Hit@1**
全部（P+S+C+F+E）	94.9%	–	0.50	91.3%
仅 P（位置）	87.3%	-7.6	0.60	93.5%
仅 S（结构）	34.5%	-60.4	**0.70**	**94.9%**
仅C（内容）	28.7%	-66.2	0.80	93.8%
仅F（流程）	15.2%	-79.7	0.90	91.1%
仅E	12.1%	-82.8		
P+S	92.4%	-2.5		
P+C	90.9%	-4.0		
P+F（流程）/ P+E（置信度）	89.5 / 88.7%	-5.4 / -6.2		
S+C / S+F	45.6 / 38.2%	-49.3 / -56.7		
P（位置）+S（结构）+C（内容）	93.8%	-1.1		
P（位置）+S（结构）+C（内容）+F（流程）	94.5%	-0.4		

表5：鲁棒性分析：按缺陷类型、轨迹长度和错误位置分层的Hit@1和平均倒数排名。

分层	**类别**	**Hit@1**	**MRR**
缺陷类型	逻辑错误	95.8%	0.98
	通信失败	93.6%	0.96
	数据损坏	94.5%	0.97
	缺失验证	94.3%	0.96
	角色混淆	94.8%	0.97
轨迹长度	8–9 步骤	95.5%	0.97
	10–11 步骤	95.0%	0.97
	12–13 步骤	94.2%	0.96
	14–15 步骤	93.9%	0.96
错误位置	早期（步骤 2–3）	96.1%	0.98
	中间阶段（步骤4–6）	93.3%	0.96
	后期阶段（步骤 7+）	92.7%	0.95

示例1：软件开发。任务：实现平均值计算。

步骤 1 [规划器]：分析需求 -> “求和所有数字，统计元素数量，进行除法” 步骤 2 [编码器]：编写实现 -> “def average(nums): ...” 步骤 3 [编码器]：添加边界情况处理 [BUG]->“if nums = []: # BUG: =而非 ==” 步骤 4 [审查员]：审查代码 -> “代码看起来是正确的。” 步骤 5 [执行器]：运行测试 [ERROR]-> “SyntaxError: 第2行语法无效”

根本原因：步骤3（逻辑错误）。AGENTTRACE 正确识别了步骤3；大语言模型基线选择了步骤5（错误节点），这说明了其倾向于选择错误表现，而非回溯溯源。

示例2：ResearchAnalysis。任务：分析Transformer架构出版物。

步骤 1 [搜索器]：查询数据库 -> “150 篇论文” 步骤 2 [搜索器]：过滤 -> “25 篇相关论文” 步骤 3 [分析器]：提取发现 [缺陷]-> “注意力机制被MLP取代”（实际情况为：注意力机制被增强） 步骤 4 [综合器]：“该领域正在远离

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

```
    attention-based models..."
Step 5 [Writer]: "Transformers becoming obsolete"
Step 6 [Writer]: Finalize report  [ERROR]
  -> "Contradicts recent SOTA results"
```

基于注意力的模型……” 步骤 5 [撰写器]：“Transformer（变形器）即将过时” 步骤 6 [撰写器]：定稿报告 [ERROR]> “与最新最先进（State of the Art）结果相矛盾”

Root cause: Step 3 (communication failure). AGENTTRACE correctly identifies Step 3; the LLM selects Step 4 (propagation step), demonstrating the challenge of distinguishing root causes from their downstream effects.

根本原因：步骤3（通信失败）。AGENTTRACE正确识别了步骤3；大语言模型选择了步骤4（传播步骤），这体现了区分根本原因与其下游影响的挑战性。

5.7 RUNTIME PERFORMANCE

AGENTTRACE processes traces in 0.12 seconds on average, compared to 8.3 seconds for LLM-based analysis. This 69× speedup enables interactive debugging workflows.

5.7 运行时性能

AGENTTRACE处理执行轨迹的平均时间为0.12秒，而基于大语言模型的分析需要8.3秒。这一 69× 加速实现了交互式调试工作流。

6 DISCUSSION

Why Position Features Dominate. Importantly, the dominance of positional signals should not be interpreted solely as a benchmark artifact. In many deployed agent systems, early planning or routing decisions shape the entire downstream execution, making early-stage errors disproportionately impactful. Our analysis reveals that bugs occurring earlier in execution traces tend to cause errors that manifest later—a pattern that reflects a fundamental property of hierarchical multi-agent workflows where upstream decisions constrain downstream actions.

6 讨论

为何位置特征占主导地位。重要的是，位置信号的主导地位不应仅被解释为基准测试伪影。在许多已部署的智能体系统中，早期规划或路由决策塑造了整个下游执行过程，使得早期阶段的错误具有不成比例的影响。我们的分析表明，发生在执行轨迹较前期的缺陷往往会导致后期显现的错误——这一模式反映了分层多智能体工作流的一个基本属性：上游决策约束下游动作。

Failure Case Analysis. Among the 28 scenarios (5.1%) where AGENTTRACE failed to rank the correct root cause first, the dominant failure mode is *multiple plausible root causes* (12 cases, 42.9%): e.g., in scenario `res_3867e9`, both Step 2 (incorrect search query) and Step 3 (overly strict filter) contributed to the error. The second mode is *bugs at unusual positions* (8 cases, 28.6%): late-stage bugs (step 7+) have feature distributions that differ from the majority, causing AGENTTRACE to over-prioritize earlier steps. The remaining failures stem from atypical causal structures (5 cases) and feature extraction issues (3 cases). Potential improvements include position-adaptive weights and multi-root-cause detection.

失败案例分析。在AGENTTRACE未能将正确的根本原因排在首位的28个场景（占5.1%）中，主要的失败模式是存在多个可能的根本原因（12例，占42.9%）：例如，在场景 res 3867e9 中，步骤2（错误的搜索查询）和步骤3（过于严格的过滤器）共同导致了错误。第二种模式是异常位置处的缺陷（8例，占28.6%）：后期阶段的错误（步骤 7+）具有与大多数步骤不同的特征分布，导致AGENTTRACE过度优先考虑较早的步骤。其余的失败源于非典型因果结构（5例）和特征提取问题（3例）。潜在的改进方向包括位置自适应权重和多根本原因检测。

Limitations. Our current evaluation focuses on synthetic scenarios with single root causes. Real multi-agent failures often involve multiple contributing factors and more complex causal structures. Additionally, our benchmark assumes accurate execution logging, which may not always be available in production systems.

局限性。我们当前的评估聚焦于具有单一根本原因的合成场景。实际的多智能体故障往往涉及多个促成因素以及更复杂的因果结构。此外，我们的基准测试假设执行日志准确无误，而这在生产系统中并不总是可用。

Implications for Agent Safety. As multi-agent systems are deployed in high-stakes domains, the ability to quickly identify and understand failures becomes critical for maintaining trust and safety. AGENTTRACE provides a foundation for post-hoc analysis that can inform both immediate fixes and systematic improvements to agent design.

对智能体安全的影响。随着多智能体系统在高风险领域中的部署，快速识别和理解故障的能力对于维护信任和安全变得至关重要。AGENTTRACE 为事后分析提供了基础，可以为智能体设计的即时修复和系统性改进提供信息支持。

7 CONCLUSION

We presented AGENTTRACE, a causal graph-based framework for root cause localization in deployed multi-agent systems. Our approach achieves high accuracy on a diverse benchmark while maintaining sub-second latency. The framework's reliance on interpretable structural and positional features—rather than expensive LLM inference—makes it practical for interactive debugging in production environments. Future work will extend the approach to handle multiple concurrent root causes and validate on production traces. We hope AGENTTRACE can serve as a practical diagnostic layer for agentic systems deployed in real-world, reliability-critical environments.

7 结论

我们提出了AGENTTRACE，一个用于已部署多智能体系统中根本原因定位的基于因果图的框架。我们的方法在多样化基准测试上实现了高准确率，同时保持了亚秒级延迟。该框架依赖可解释的结构和位置特征，而非昂贵的大语言模型推理，这使其在生产环境中用于交互式调试具有实用性。未来的工作将扩展该方法以处理多个并发根本原因，并在生产轨迹上进行验证。我们希望AGENTTRACE能够作为部署在真实世界可靠性关键环境中的智能体系统的一个实用诊断层。

REPRODUCIBILITY STATEMENT

Our benchmark generation code, evaluation scripts, and all experimental results are available at `https://github.com/GeoffreyWang1117/AgentTrace/tree/iclr2026-aiwild-camera-ready`. The benchmark includes complete execution traces, ground truth annotations, and baseline implementations.

可重复性声明

我们的基准生成代码、评估脚本以及所有实验结果均可在 https://github.com/GeoffreyWang1117/AgentTrace/tree/ iclr2026-aiwild-camera-ready 获取。该基准包含完整的执行轨迹、真实值标注以及基线实现。

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

REFERENCES

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations*, 2024.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). In *International Conference on Machine Learning*, pp. 2668–2677, 2018.

Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. In *Stanford InfoLab Technical Report*, 1999.

Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. Exploring llm-based agents for root cause analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024.

Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. In *ICLR Workshop*, 2014.

Jacopo Soldani and Antonio Brogi. Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. *ACM Computing Surveys*, 55(3):1–39, 2022.

Uber Technologies. Jaeger: Open source, end-to-end distributed tracing. `https://www.jaegertracing.io/`, 2017.

Twitter. Zipkin: A distributed tracing system. `https://zipkin.io/`, 2012.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A BENCHMARK GENERATION DETAILS

Each scenario in our benchmark simulates a multi-agent system with 3–5 specialized agents: a **Coordinator** (task decomposition and delegation), **Specialists** (domain-specific tasks), a **Reviewer** (output validation), and an **Executor** (final actions). Table 6 lists the domain-specific configurations.

Each scenario follows a standardized JSON schema encoding the execution steps, agent metadata, and ground truth annotations:

```
{ "scenario_id": "cod_034b23", "domain": "coding",
  "agents": ["Planner","Coder","Reviewer","Executor"],
  "steps": [{"step_id": 1, "agent": "Planner",
    "action_type": "plan", "input": "...",
    "output": "...", "timestamp": "..."}, ...],
  "ground_truth": {"error_node_id": 8,
    "root_cause_node_id": 3, "bug_type": "logic_error",
    "bug_description": "Incorrect comparison operator"}}
```

参考文献

陈信云，林马克斯韦尔，Nathanael Sch¨arli，以及周丹尼。教导大型语言模型自我调试。载于国际学习表征会议，2024 年。

洪思睿，诸葛明辰，乔纳森·陈，郑夏武，程宇衡，张策耀，王进林，王梓丽，余世雄，林子娟，等。MetaGPT：多智能体协作框架的元编程。发表于2024年国际学习表征会议。

金彬，马丁·瓦滕贝格，贾斯汀·吉尔默，蔡凯丽，詹姆斯·韦克斯勒，费尔南达·维加，罗里·赛斯。超越特征归因的可解释性：使用概念激活向量（TCAV）进行定量测试。发表于2018年国际机器学习会议，第2668–2677页。

劳伦斯·佩奇，谢尔盖·布林，拉吉夫·莫特瓦尼，特里·温诺格拉德。PageRank引用排序：为网络带来秩序。斯坦福信息实验室技术报告，1999年。

德夫吉特·罗伊，张旭超，拉西·巴韦，切坦·班萨尔，佩德罗·拉斯-卡萨斯，罗德里戈·丰塞卡，萨拉万·拉杰莫汉。探索基于大语言模型的智能体进行根本原因分析。发表于第32届ACM国际软件工程质量基础会议附录，2024年。

卡伦·西蒙尼扬，安德烈亚·维达利，安德鲁·齐瑟曼。深入卷积神经网络内部：可视化图像分类模型和显著性图。国际学习表征会议研讨会，2014。

贾科波·索达尼，安东尼奥·布罗吉。（微）服务云应用中的异常检测与故障根本原因分析：综述。ACM计算综述，55(3):1–39，2022。

Uber科技。Jaeger：开源端到端分布式追踪。https://www. jaegertracing.io/，2017。

推特。Zipkin：一个分布式追踪系统。https://zipkin.io/，2012。

魏哲生，王学志，戴尔·舒尔曼斯，马滕·博斯玛，布莱恩·伊赫特，夏飞，奇·埃德，李庆华，周丹尼。思维链提示诱导大型语言模型进行推理。Advancesin Neural InformationProcessing Systems，35:24824–24837，2022。

吴青云, 加根·班萨尔, 张杰宇, 吴亦燃, 李贝贝, 朱尔康, 蒋丽, 张晓芸, 王驰. AutoGen：通过多智能体对话实现下一代大语言模型应用。arXiv预印本 arXiv:2308.08155，2023年。

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, 曹远. React：在语言模型中协同推理与行动。载于《国际学习表征会议》，2023年。

一个基准测试生成细节

我们基准测试中的每个场景都模拟了一个由3至5个专业化智能体组成的多智能体系统：一个协调者（负责任务分解和委派）、专家（负责领域特定任务）、一个审查员（负责输出验证）和一个执行器（负责最终动作）。表6列出了领域特定配置。

每个场景都遵循标准化的JSON模式，编码了执行步骤、智能体元数据和地面真实注释：

{ "scenario_id": "cod_034b23", "domain": "coding", "agents": ["规划器","编码器","审查员","执行器"],"步骤": [{"步骤_id": 1, "agent": "规划器", "action_type": "计划", "输入": "...", "输出": "...", "时间戳": "..."}, ...],"ground_truth": {"error_node_id": 8, "root_cause_node_id": 3,"缺陷_类型": "逻辑_错误", "缺陷_描述": "不正确的比较运算符"}}

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

Table 6: Domain-specific agent configurations

Domain	Agents	Interaction Pattern	N
Software Dev	Planner, Coder, Reviewer, Executor	Sequential + Review Loop	52
Customer Service	Router, Specialist, Resolver, Logger	Hierarchical Dispatch	51
Research	Searcher, Analyzer, Synthesizer, Writer	Pipeline + Feedback	51
Planning	Scheduler, Optimizer, Validator, Notifier	Iterative Refinement	46
Trading	Analyst, Strategist, RiskManager, Executor	Parallel Analysis	50
Healthcare	Triager, Specialist, Pharmacist, Coordinator	Consultation Chain	60
Legal	Researcher, Analyst, Drafter, Reviewer	Document Pipeline	60
Education	Assessor, Tutor, ContentGenerator, Evaluator	Adaptive Loop	60
Finance	DataCollector, Analyst, Advisor, Reporter	Aggregation Pattern	60
DevOps	Monitor, Diagnoser, Remediator, Verifier	Incident Response	60

B Feature Weight Learning

Feature group weights were determined through grid search on a held-out validation set of 50 scenarios (not included in the 550-scenario benchmark):

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \sum_{i=1}^{50} \mathbb{1}[\text{rank}(v_{\text{root}}^{(i)}|\mathbf{w}) = 1] \tag{3}$$

The search space was $w_p \in \{0.5, 0.6, 0.7, 0.8\}$, $w_s \in \{0.1, 0.15, 0.2, 0.25\}$, $w_c \in \{0.03, 0.05, 0.07, 0.1\}$, $w_f \in \{0.02, 0.03, 0.05\}$, $w_e \in \{0.01, 0.02, 0.03\}$, subject to $\sum_i w_i = 1$.

C Statistical Analysis Details

Bootstrap Confidence Intervals. We compute 95% CIs using bootstrap resampling with B=10,000 iterations. For each bootstrap sample $\mathbf{r}^{(b)}$, we compute $\hat{\theta}^{(b)} = \text{mean}(\mathbf{r}^{(b)})$, then take the 2.5th and 97.5th percentiles as interval bounds.

McNemar's Test. For comparing two methods A and B, we construct a 2×2 contingency table of concordant/discordant predictions. The test statistic is $\chi^2 = (|n_{01} - n_{10}| - 1)^2/(n_{01} + n_{10})$, following a χ^2 distribution with 1 degree of freedom.

Effect Size. We use Cohen's $h = 2\arcsin(\sqrt{p_1}) - 2\arcsin(\sqrt{p_2})$, where $|h| < 0.2$ is small, $0.2 \leq |h| < 0.5$ is medium, and $|h| \geq 0.5$ is a large effect.

D Detailed Per-Domain Results

Table 7: Detailed performance metrics by domain

Domain	N	Hit@1	Hit@3	Hit@5	MRR
Software Development	52	94.2%	96.2%	100.0%	0.96
Customer Service	51	92.2%	96.1%	100.0%	0.94
Research Analysis	51	96.1%	98.0%	100.0%	0.97
Planning & Scheduling	46	91.3%	95.7%	100.0%	0.94
Financial Trading	50	90.0%	98.0%	100.0%	0.94
Healthcare Coordination	60	95.0%	100.0%	100.0%	0.97
Legal Document Analysis	60	100.0%	100.0%	100.0%	1.00
Educational Tutoring	60	93.3%	100.0%	100.0%	0.96
Financial Advisory	60	96.7%	100.0%	100.0%	0.98
DevOps Automation	60	98.3%	98.3%	100.0%	0.99
Overall	**550**	**94.9%**	**98.4%**	**99.8%**	**0.97**

表6：领域特定的智能体配置

领域	智能体	交互模式	N
软件开发	规划器, 编码器, 审查员, 执行器	顺序型 + 审查循环	52
客户服务	路由器, 专科医生, 解决器, 记录器	层次化调度	51
研究	搜索器、分析器、综合器、撰写器	流水线 + 反馈	51
规划	调度器、优化器、验证器、通知器	迭代优化	46
交易	分析师, 策略师, 风险管理员, 执行器	并行分析	50
医疗保健	分诊员, 专科医生, 药剂师, 协调者	咨询链	60
法律	研究员, 分析师, 起草人, 审查员	文档流水线	60
教育	评估器, 导师, 内容生成器, 评价器	自适应循环	60
金融	数据收集器, 分析师, 顾问, 报告员	聚合模式	60
DevOps	监控器, 诊断器, 修复器, 验证器	事件响应	60

B 特征权重学习

特征组权重通过在一个包含50个场景的留出验证集（未包含在550个场景的基准测试中）上进行网格搜索来确定：

$$\mathbf{w}^* = \arg\max_{\mathbf{w}} \sum_{i=1}^{50} \mathbb{1}[\text{rank}(v_{\text{root}}^{(i)}|\mathbf{w}) = 1] \tag{3}$$

搜索空间为 $w_p \in \{0.5, 0.6, 0.7, 0.8\}$、$w_s \in \{0.1, 0.15, 0.2, 0.25\}$、$w_c \in \{0.03, 0.05, 0.07, 0.1\}$、$w_f \in \{0.02, 0.03, 0.05\}$、$w_e \in \{0.01, 0.02, 0.03\}$，受 $\sum_i w_i = 1$约束。

C（内容）统计分析细节

自举置信区间。我们使用包含B=10,000次迭代的自举重采样方法计算95%置信区间。对于每个自举样本$\mathbf{r}^{(b)}$，我们计算$\hat{\theta}^{(b)} = \text{mean}(\mathbf{r}^{(b)})$，然后取第2.5和第97.5百分位数作为区间边界。

麦纳玛检验。为比较两种方法A和B，我们构建一致/不一致预测的列联表。检验统计量为 $\chi^2 = (|n_{01} - n_{10}| - 1)^2/(n_{01} + n_{10})$，服从自由度为1的 χ^2 分布。

效应量。我们使用Cohen's $h = 2\arcsin(\sqrt{p_1}) - 2\arcsin(\sqrt{p_2})$，其中 $|h| < 0.2$ 表示小效应，$0.2 \leq |h| < 0.5$ 表示中等效应，$|h| \geq 0.5$ 表示大效应。

D 详细分领域结果

表 7：按领域划分的详细性能指标

领域	N	Hit@1	Hit@3	Hit@5	MRR
软件开发	52	94.2%	96.2%	100.0%	0.96
客户服务	51	92.2%	96.1%	100.0%	0.94
研究分析	51	96.1%	98.0%	100.0%	0.97
计划与调度	46	91.3%	95.7%	100.0%	0.94
金融交易	50	90.0%	98.0%	100.0%	0.94
医疗协调	60	95.0%	100.0%	100.0%	0.97
法律文件分析	60	100.0%	100.0%	100.0%	1.00
教育辅导	60	93.3%	100.0%	100.0%	0.96
财务咨询	60	96.7%	100.0%	100.0%	0.98
DevOps自动化	60	98.3%	98.3%	100.0%	0.99
总体	**550**	**94.9%**	**98.4%**	**99.8%**	**0.97**

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

E LLM BASELINE DETAILS

We use GPT-4 (`gpt-4-0613`) with temperature 0.0, max tokens 1024, top-p 1.0, and zero frequency/presence penalties. The prompt instructs the model to analyze the full execution trace, identify causal relationships, and output only the step number of the root cause:

```
You are an expert debugger analyzing a multi-agent system
execution trace. The system encountered an error.
Your task is to identify the ROOT CAUSE - the earliest
step where something went wrong that led to the final error.
## Execution Trace:
{trace_content}
## Error Description:
The system failed at step {error_step}: {error_description}
## Instructions:
1. Analyze the execution trace carefully
2. Identify causal relationships between steps
3. Find the EARLIEST step that caused the error
4. Consider: logic errors, miscommunication,
   data issues, missing validation
## Output Format:
Respond with ONLY the step number, e.g., "3"
Root cause step:
```

Table 8 categorizes the 173 errors made by the LLM baseline. The most common failure mode (47.4%) is selecting the error manifestation node rather than tracing back to the actual root cause.

Table 8: Error analysis for LLM baseline

Error Type	Count	%
Selected error node	82	47.4
Off-by-one (adjacent)	45	26.0
Intermediate step	31	17.9
Completely incorrect	15	8.7
Total	**173**	**100**

F RUNTIME PERFORMANCE DETAILS

Tables 9 and 10 break down runtime by component and show scaling with trace length. Runtime scales approximately linearly: $T \approx 12.5n + 5$ ms, where n is the number of steps. All experiments were conducted on an AMD Ryzen 9 5950X (16-core, 3.4 GHz), 128 GB DDR4 RAM, NVMe SSD, Ubuntu 24.04 LTS, Python 3.13.

Table 9: Component-wise runtime breakdown

Component	Mean (ms)	Std (ms)
Graph Construction	15.2	3.1
Backward Tracing	8.4	2.8
Feature Extraction	62.3	12.4
Node Ranking	28.6	5.2
Total	**114.5**	**18.3**

Table 10: Runtime scaling with trace length

Steps	Mean (ms)	P95 (ms)
5	68	89
10	115	148
15	162	201
20	218	267
25	283	342

G REPRODUCIBILITY

All code, data, and evaluation scripts are available at `https://github.com/GeoffreyWang1117/AgentTrace/tree/iclr2026-aiwild-camera-ready`,

E 大语言模型基线细节

我们使用温度设为0.0、最大令牌数为1024、top-p 1.0，且频率/存在性惩罚为零的GPT-4 (gpt-4-0613)。该提示要求模型分析完整的执行轨迹，识别因果关系，并仅输出根本原因对应的步骤编号：

```
你是一名分析多智能体系统执行轨迹的专家调试员。该系统遇到了错误。你的任
务是识别根本原因——导致最终错误的最早步骤。## 执行轨迹：{trace_
content} ## 错误描述：系统在步骤 {error_step} 失败：{error_description} #
# 说明：1. 仔细分析执行轨迹 2. 识别步骤之间的因果关系 3. 找到导致错误的最早
步骤 4. 考虑：逻辑错误、通信失败、数据问题、缺失验证

## 输出格式：仅以步骤编号作为响应，例如 "3" 根本原因
步骤：
```

表8对LLM基线中出现的173个错误进行了分类。最常见的故障模式（47.4%）是选择错误表现节点，而不是追溯到真正的根本原因。

表 8：LLM基线的错误分析

错误类型	数量	%
选定的错误节点	82	47.4
偏差一（相邻）	45	26.0
中间步骤	31	17.9
完全错误	15	8.7
总计	**173**	**100**

F（流程） 运行时性能详情

表 9 和 表 10 按组件划分了运行时，并展示了其随轨迹长度的扩展情况。运行时近似线性扩展：$T \approx 12.5n + 5$ms，其中 n 是步骤数。所有实验均在 AMD Ryzen 9 5950X（16 核，3.4 GHz）、128 GB DDR4 内存、NVMe 固态硬盘、Ubuntu 24.04 LTS、Python 3.13 上进行。

表 9：按组件划分的运行时分解

组件	均值 (毫秒)	标准差 (毫秒)
图构建	15.2	3.1
向后追踪	8.4	2.8
特征提取	62.3	12.4
节点排序	28.6	5.2
总计	**114.5**	**18.3**

表10：轨迹长度与运行时扩展

步骤	均值 (毫秒)	P95（毫秒）
5	68	89
10	115	148
15	162	201
20	218	267
25	283	342

G 可复现性

所有代码、数据和评估脚本均可在
https://github.com/GeoffreyWang1117/AgentTrace/tree/ICLR 2026-aiwild-最终版 获取,

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next

including benchmark generation scripts, 550 scenario JSON files with ground truth, all baseline implementations, and statistical analysis scripts. Table 11 lists all hyperparameters. Random seeds: benchmark generation (42), bootstrap resampling (12345), train/validation split (2024).

Table 11: Complete hyperparameter settings

Parameter	**Value**
Max tracing depth	10
w_p / w_s / w_c	0.70 / 0.20 / 0.05
w_f / w_e	0.03 / 0.02
Normalization ϵ	10^{-8}
Bootstrap iters	10,000
Confidence level	95%

包括基准测试生成脚本、550个带有真实标签的场景JSON文件、所有基线实现以及统计分析脚本。表11列出了所有超参数。随机种子：基准测试生成（42）、自举重采样（12345）、训练/验证划分（2024）。

表11：完整的超参数设置

参数	**值**
最大追踪深度	10
w_p / w_s / w^c	0.70 / 0.20 / 0.05
w_f / w_e	0.03 / 0.02
归一化 ϵ	10^{-8}
自助迭代次数	10,000
置信水平	95%

翻译服务由 ChatECNU 大语言模型平台提供 https://chat.ecnu.edu.cn/html/
技术方案基于 PDFMathTranslate-next 项目 https://github.com/PDFMathTranslate/PDFMathTranslate-next